

Toán cho thuật toán (The Mathematics You Actually Reuse)

PHỤ THUỘC A/03 cho ký hiệu tiệm cận. Chương này không nên đọc tuần tự: đọc lướt một lượt để biết trong đó có gì, rồi quay lại đúng mục khi một chương sau cần tới.

TÓM TẮT

Đây là kho toán dùng lại nhiều nhất trong 23 chương còn lại, chia theo *câu hỏi nó trả lời* chứ không theo tên môn. Mục 1 là tổng và chặn — dùng mỗi lần bạn phân tích một vòng lặp. Mục 2 là tổ hợp đếm — dùng cho mọi bài đếm và cho việc ước lượng kích thước không gian tìm kiếm. Mục 3 là xác suất và kỳ vọng, với một công cụ mạnh bất thường là kỳ vọng tuyến tính. Mục 4 là số học modulo, mục 5 là đại số rời rạc, mục 6 là lý thuyết trò chơi tối thiểu. Nếu chỉ nhớ một điều: ba thứ quay lại nhiều nhất là chuỗi hình học bị chặn bởi hai lần số hạng lớn nhất, tổng điều hoà xấp xỉ $\ln n$, và kỳ vọng của tổng bằng tổng các kỳ vọng dù các biến phụ thuộc nhau.

KIẾN THỨC CẦN CÓ TRƯỚC

- **chuỗi hình học** — tổng dạng $1 + r + r^2 + \dots$; khi $r > 1$, tổng bị chi phối bởi số hạng cuối, khi $r < 1$ thì bởi số hạng đầu.
- **tổng điều hoà** — $H_n = 1 + 1/2 + \dots + 1/n \approx \ln n + 0,577$. Xuất hiện mỗi khi có “lần thứ i tốn $1/i$ ” hoặc “đếm bội số”.
- **hệ số nhị thức** — $\binom{n}{k}$ — số cách chọn k phần tử từ n phần tử, không kể thứ tự.
- **biến chỉ báo (indicator)** — biến nhận giá trị 1 nếu một sự kiện xảy ra và 0 nếu không. Kỳ vọng của nó bằng đúng xác suất của sự kiện.
- **số học modulo** — làm việc với phần dư khi chia cho m ; phép cộng, trừ, nhân đều bảo toàn phần dư, phép chia thì cần nghịch đảo.
- **nguyên lý chuồng bồ câu** — nhốt $n + 1$ con vào n chuồng thì có chuồng chứa từ hai con trở lên. Công cụ chứng minh tồn tại đơn giản nhất.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao chuỗi $1 + 2 + 4 + \dots + n$ lại “chỉ bằng” $O(n)$, trong khi chuỗi $1 + 1/2 + \dots + 1/n$ lại không phải $O(1)$?
- Q2. Kỳ vọng của tổng bằng tổng các kỳ vọng ngay cả khi các biến phụ thuộc chằng chịt. Vì sao điều đó lại đúng, và vì sao nó mạnh đến vậy?
- Q3. Vì sao gần như mọi bài đếm trong lập trình thi đấu đều yêu cầu kết quả theo modulo một số nguyên tố lớn?
- Q4. Nguyên lý chuồng bồ câu tầm thường tới mức trẻ con cũng hiểu. Vì sao nó lại chứng minh được những điều không hiển nhiên chút nào?
- Q5. Khi nào một bài toán “tính giá trị sau 10^{18} bước” lại giải được, và cấu trúc nào làm nó giải được?

Chương này khác mọi chương khác trong quyển sách: nó không có một mạch lập luận từ đầu tới cuối, vì nó là kho chứ không phải là bài. Cách dùng đúng là đọc lướt một lượt ngay bây giờ —

đủ để nhớ “trong đó có gì” — rồi quay lại từng mục khi một chương sau thực sự cần. Mỗi mục đều ghi rõ nó quay lại ở chương nào.

Nguyên tắc chọn nội dung cũng cần nói rõ: tôi chỉ giữ những mảnh toán mà tôi thấy quay lại *ít nhất ba lần* trong các chương sau. Rất nhiều thứ đẹp đẽ đã bị bỏ ra ngoài, không phải vì chúng không hay mà vì học một công cụ không có nhu cầu đi kèm thì hai tuần sau sẽ quên sạch.

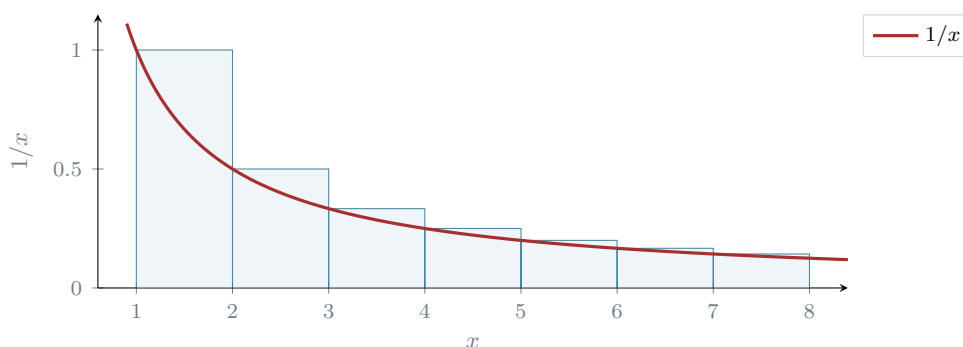
1.1 Tổng và chặn: đọc chi phí của một vòng lặp

Ba dạng tổng chiếm gần hết những gì bạn cần.

Cấp số cộng. $1 + 2 + \dots + n = n(n+1)/2 = \Theta(n^2)$. Xuất hiện khi vòng lặp trong chạy tới chỉ số của vòng lặp ngoài — ví dụ mọi thuật toán “xét mọi cặp”.

Cấp số nhân. $1 + 2 + 4 + \dots + 2^k = 2^{k+1} - 1 < 2 \cdot 2^k$. Điều đáng nhớ không phải công thức mà là câu này: **tổng của một cấp số nhân tăng bị chặn bởi hai lần số hạng lớn nhất**. Câu đó giải thích ngay chi phí khấu hao của mảng động (A/03), chi phí xây dựng cây phân đoạn, và chi phí tổng cộng của kỹ thuật nhân đôi. Ngược lại, với $r < 1$ thì tổng bị chặn bởi hằng số nhân số hạng đầu: $1 + 1/2 + 1/4 + \dots < 2$ — lý do vì sao chia đôi mãi vẫn chỉ tốn tuyến tính.

Tổng điều hoà. $H_n = \sum_{i=1}^n 1/i \approx \ln n$. Đây là tổng bị đánh giá sai nhiều nhất: nó *không* hội tụ, nhưng tăng cực chậm. Hình 1.1 cho thấy vì sao xấp xỉ bằng $\ln n$: tổng các cột chữ nhật kẹp giữa hai tích phân. Tổng này xuất hiện ở ba chỗ rất khác nhau và đáng nhớ cả ba: sàng Eratosthenes tốn $\sum_p n/p \approx n \ln \ln n$ (D/21); quicksort ngẫu nhiên tốn trung bình $2n \ln n$ so sánh (C/14); và bài toán “sưu tập đủ bộ” cần trung bình nH_n lần thử.



Hình 1.1. Vì sao $H_n \approx \ln n$. Mỗi cột có diện tích $1/i$, nên tổng diện tích các cột là H_n ; đường cong $1/x$ có tích phân từ 1 tới n bằng $\ln n$. Các cột kẹp đường cong từ trên, cho $\ln n < H_n < \ln n + 1$. Bài học tổng quát hơn: khi cần chặn một tổng, hãy so nó với tích phân của hàm tương ứng — đây là mẹo dùng được cho hầu hết các tổng đơn điệu.

Ngoài ba dạng trên, hai xấp xỉ đáng nhớ: $\log_2(n!) = \Theta(n \log n)$ — công thức Stirling, và là lý do cận dưới của sắp xếp so sánh là $\Omega(n \log n)$ (D/24); và $\binom{n}{k} \leq (en/k)^k$ — dùng để chặn kích thước không gian tìm kiếm.

1.2 Tổ hợp đếm: bao nhiêu khả năng, và đếm chúng thế nào

Bốn công thức nền, mỗi công thức ứng với một câu hỏi.

Chọn có thứ tự, không lặp: $n!/(n-k)!$. Chọn không thứ tự, không lặp: $\binom{n}{k}$. Chọn có lặp, có thứ tự: n^k . Chọn có lặp, không thứ tự (bài toán chia kẹo): $\binom{n+k-1}{k}$ — kỹ thuật “sao và vách ngăn”.

Hai đẳng thức đáng thuộc vì chúng biến thành thuật toán. *Đẳng thức Pascal* $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ chính là công thức truy hồi để tính bảng hệ số nhị thức bằng quy hoạch động, và cách chứng minh nó — “phần tử thứ n hoặc được chọn hoặc không” — là khuôn mẫu của mọi lập luận chia trường hợp trong chương C/16. *Nguyên lý bao hàm–loại trừ* cho phép đếm hợp của các tập bằng cách cộng rồi trừ các giao; nó là nền của kỹ thuật đếm bù — đếm cái xấu rồi lấy tổng trừ đi — vốn thường dễ hơn đếm trực tiếp cái tốt.

Số Catalan $C_n = \frac{1}{n+1} \binom{2n}{n}$ đáng biết vì nó đếm một họ bài toán trông rất khác nhau nhưng cùng cấu trúc: số dãy ngoặc đúng độ dài $2n$, số cây nhị phân có n nút, số cách chia đa giác thành tam giác, số đường đi không vượt đường chéo. Khi bạn tính tay các trường hợp nhỏ và ra dãy 1, 2, 5, 14, 42 thì gần như chắc chắn bài của bạn có cấu trúc Catalan — một ví dụ đẹp cho heuristic “làm ví dụ nhỏ rồi tra dãy số” ở chương A/01.

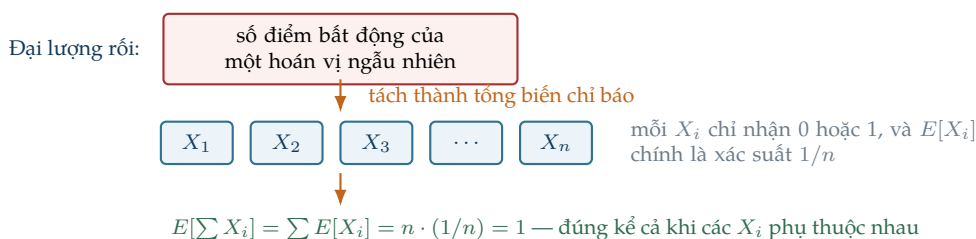
Nguyên lý chuồng bồ câu tầm thường về phát biểu nhưng mạnh về hệ quả, vì nó chứng minh sự tồn tại mà không cần chỉ ra vật thể. Ví dụ dùng trong thuật toán: trong $n+1$ số bất kỳ từ 1 tới $2n$, luôn có hai số mà số này chia hết số kia; hoặc trong một dãy n^2+1 số phân biệt luôn có dãy con đơn điệu độ dài $n+1$. Cách dùng thực dụng nhất: chứng minh rằng một cấu trúc lặp lại phải xuất hiện sau một số bước nhất định — đó chính là cơ sở của thuật toán tìm chu trình ở chương C/20.

1.3 Xác suất và kỳ vọng: công cụ mạnh nhất là công cụ đơn giản nhất

Nếu chỉ được giữ một công cụ xác suất cho thuật toán, đó là *kỳ vọng tuyến tính*: kỳ vọng của tổng bằng tổng các kỳ vọng, $E[X+Y] = E[X] + E[Y]$, **kể cả khi X và Y phụ thuộc nhau**. Câu cuối là chỗ làm nó mạnh, và cũng là chỗ người mới không tin.

Cách dùng chuẩn gồm ba bước: viết đại lượng cần tính thành *tổng của các biến chỉ báo*; tính kỳ vọng của từng biến chỉ báo, mà kỳ vọng đó chỉ là xác suất; rồi cộng lại. Ví dụ: số điểm bất động trung bình của một hoán vị ngẫu nhiên. Đặt $X_i = 1$ nếu phần tử i đứng đúng chỗ. Ta có $E[X_i] = 1/n$, và các X_i rõ ràng phụ thuộc nhau — nhưng không sao: $E[\sum X_i] = n \cdot (1/n) = 1$. Trung bình đúng một điểm bất động, bất kể n lớn thế nào. Tính trực tiếp bằng cách xét phân phối thì rắc rối hơn nhiều bậc.

Cùng kỹ thuật này giải thích chi phí trung bình của quicksort ngẫu nhiên: đặt biến chỉ báo cho sự kiện “phần tử thứ i và thứ j từng được so sánh với nhau”, tính xác suất của nó là $2/(|i-j|+1)$, rồi cộng — và tổng điều hoà xuất hiện, cho $\Theta(n \log n)$.



Hình 1.2. Quy trình ba bước của kỳ vọng tuyến tính. Điểm làm nó mạnh bất thường: phép cộng kỳ vọng không cần bất kỳ giả thiết độc lập nào, nên một đại lượng có phân phối rất rối vẫn tính được bằng cách chẻ thành nhiều câu hỏi có/không rất dễ.

Ba bất đẳng thức đuôi, xếp theo thứ tự cần thêm giả thiết và cho kết luận mạnh hơn: *Markov* chỉ cần biến không âm, cho chặn yếu; *Chebyshev* cần phương sai, cho chặn tốt hơn; *Chernoff* cần tổng các biến độc lập, và cho chặn giảm theo hàm mũ. Chernoff là lý do vì sao “lặp lại thuật toán ngẫu nhiên k lần rồi lấy đa số” làm xác suất sai giảm cực nhanh — nền của cả chương C/20.

CẦN NHỚ

Ba mẫu toán quay lại nhiều nhất trong 23 chương còn lại: (i) tổng cấp số nhân tăng $\leq$ hai lần số hạng lớn nhất; (ii) $H_n \approx \ln n$, tăng nhưng cực chậm; (iii) kỳ vọng tuyến tính đúng kể cả khi các biến phụ thuộc. Nếu chỉ mang theo ba thứ từ chương này, hãy mang ba thứ đó.

1.4 Số học modulo: vì sao mọi bài đếm đều lấy dư

Bài đếm hay cho kết quả khổng lồ — số cách xếp có thể là 2^{10^5} — nên đề bài yêu cầu kết quả theo modulo một số nguyên tố lớn, thường là $10^9 + 7$ hoặc 998 244 353. Đây không phải để làm khó mà để bài toán vẫn có ý nghĩa mà không cần số nguyên lớn.

Bốn điều cần nhớ. Cộng, trừ, nhân đều “đi xuyên qua” phép lấy dư: $(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$. Phép chia thì không, và phải thay bằng nhân với *nghịch đảo modulo*, chỉ tồn tại khi số chia nguyên tố cùng nhau với m — đây là lý do người ta chọn m nguyên tố. Lũy thừa tính bằng bình phương liên tiếp, $O(\log n)$ phép nhân. Và cẩn thận tràn số: với $m \approx 10^9$, tích hai số dư đã cỡ 10^{18} , vừa sát giới hạn của số nguyên 64 bit — một chỗ sai kinh điển. Chi tiết ở chương D/21.

1.5 Đại số rời rạc và trò chơi: hai mảnh nhỏ nhưng hay dùng

Lũy thừa ma trận biến các bài “tính trạng thái sau 10^{18} bước” thành khả thi, với điều kiện quan hệ truy hồi là *tuyến tính*: nếu $v_{k+1} = Av_k$ thì $v_n = A^n v_0$, và A^n tính được bằng bình phương liên tiếp trong $O(d^3 \log n)$. Đây là câu trả lời cho câu hỏi mở đầu thứ năm: cấu trúc làm bài giải được chính là tính tuyến tính. Cùng ý tưởng áp cho đếm đường đi độ dài k trong đồ thị, vì lũy thừa ma trận kể đếm đúng thứ đó.

Đại số trên $\mathbb{F}_2$ — cộng là phép XOR — cho phép giải hệ phương trình tuyến tính bằng các thao tác bit, rất nhanh. Ứng dụng hay gặp: tìm tập con có XOR lớn nhất, kiểm tra một số có biểu diễn được thành XOR của tập con hay không (dùng “cơ sở tuyến tính” của tập số).

Trò chơi tổ hợp. Với các trò chơi hai người, thông tin đầy đủ, không hoà, người không đi được thì thua: mỗi thế cờ là *thua* nếu mọi nước đi dẫn tới thế *thắng* cho đối thủ, và ngược lại. Với trò chơi tách được thành nhiều thành phần độc lập, định lý Sprague–Grundy nói giá trị của cả trò chơi là XOR các giá trị Grundy của từng phần — đó là lý do trò Nim giải bằng XOR các đồng. Điều đáng nhớ về mặt phương pháp: cách tìm ra quy luật này luôn là tính tay các thế nhỏ rồi tìm mẫu, đúng heuristic số một ở chương A/01.

1.6 Gốc rễ: những mẫu toán này ra đời để giải bài gì

Điều thú vị là gần như mọi mảnh toán trong chương này đều ra đời từ một bài toán rất cụ thể, thường là rất trần tục, chứ không từ nhu cầu trừu tượng.

Xác suất bắt đầu từ cờ bạc: trao đổi thư giữa Pascal và Fermat năm 1654 về cách chia tiền cược khi một ván bị bỏ dở giữa chừng. Khái niệm kỳ vọng ra đời chính từ câu hỏi đó — “ván này đáng giá bao nhiêu nếu dừng bây giờ” — và đó vẫn là cách hiểu đúng nhất về kỳ vọng khi bạn dùng nó để phân tích thuật toán ngẫu nhiên.

Tổng điều hoà và hằng số γ đi kèm đến từ Euler, trong bối cảnh tính toán các chuỗi vô hạn thế kỷ 18. Hệ số nhị thức thì cổ hơn nhiều, xuất hiện độc lập ở nhiều nền toán học — Ấn Độ, Ba Tư, Trung Quốc — trước khi mang tên Pascal ở châu Âu; tam giác Pascal được biết tới ở Trung Quốc dưới tên tam giác Dương Huy từ thế kỷ 13.

Bất đẳng thức Chernoff, thứ hiện đại nhất trong danh sách, ra đời năm 1952 trong thống kê — bài toán là kiểm định giả thuyết, cần biết xác suất một tổng lớn các quan sát lệch xa kỳ vọng nhỏ tới đâu. Ngành thuật toán mượn nó vào thập niên 1980 khi thuật toán ngẫu nhiên trở nên phổ biến, và ngày nay nó có mặt ở khắp nơi từ băm (B/08) tới học máy.

Cuối cùng, một gốc rễ đáng nhớ vì nó nói lên cách toán và thuật toán tương tác: *Concrete Mathematics*[1] của Graham, Knuth và Patashnik ra đời từ chính chương “Toán học chuẩn bị” trong TAOCP. Knuth thấy rằng để phân tích thuật toán tới hằng số, ông cần một bộ công cụ toán mà các giáo trình toán thời đó không dạy theo cách dùng được — thiên về tổng, truy hồi và tiệm cận. Nói cách khác, nhu cầu phân tích thuật toán đã tạo ra một môn toán riêng, chứ không chỉ mượn.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1654 — Pascal, Fermat	Chia tiền cược thế nào khi ván bạc bị bỏ dở.	Khái niệm kỳ vọng; nền của toàn bộ phân tích thuật toán ngẫu nhiên (C/20).
Thế kỷ 13–17 — tam giác Dương Huy / Pascal	Đếm số cách chọn trong tổ hợp và trong khai triển nhị thức.	Đẳng thức Pascal — cũng chính là công thức truy hồi quy hoạch động đầu tiên trong lịch sử (C/16).
Thế kỷ 18 — Euler	Tính các chuỗi vô hạn và xấp xỉ tổng bằng tích phân.	$H_n \approx \ln n + \gamma$; kỹ thuật chặn tổng bằng tích phân, dùng suốt trong phân tích vòng lặp.
1730 — Stirling, de Moivre	Xấp xỉ $n!$ cho các bài toán xác suất với n lớn.	$\log(n!) = \Theta(n \log n)$ — cận dưới của sắp xếp so sánh dựa trực tiếp vào đây (D/24).
1952 — Chernoff	Kiểm định giả thuyết: xác suất một tổng lệch xa kỳ vọng.	Chặn đuôi giảm theo hàm mũ; lý do “lặp lại rồi lấy đa số” hiệu quả đến thế trong thuật toán ngẫu nhiên.
1989 — Graham, Knuth, Patashnik	Phân tích thuật toán tới hằng số cần bộ công cụ toán mà giáo trình thời đó không dạy theo cách dùng được.	<i>Concrete Mathematics</i> : tổng, truy hồi, tiệm cận trở thành một môn riêng sinh ra từ nhu cầu của ngành thuật toán.

1.7 Liên hệ ngang: cùng công cụ, khác miền

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Thống kê và học máy	Bất đẳng thức tập trung, cận tổng quát hoá, biến chỉ báo	Cùng bộ công cụ đuôi (Markov, Chebyshev, Chernoff), chỉ khác câu hỏi: ở đây ta chặn xác suất thuật toán chạy chậm, ở đó chặn xác suất mô hình sai trên dữ liệu mới.
Vật lý thống kê	Hàm phân hoạch, đếm trạng thái vi mô	Cả hai đều đếm số cấu hình rồi lấy tổng có trọng số. Kỹ thuật “đếm bù” và bao hàm–loại trừ xuất hiện ở cả hai. Khác: vật lý quan tâm giới hạn nhiệt động, thuật toán quan tâm chi phí tính chính xác con số đó.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Lý thuyết thông tin	Entropy, cận dưới cho mã hoá	$\log_2(\text{số khả năng})$ là số bit tối thiểu; đây chính là dạng thông tin của cận dưới $\Omega(n \log n)$ cho sắp xếp so sánh (D/24). Cùng một phép đếm, hai cách phát biểu.
Mật mã	Số học modulo, lũy thừa nhanh, nghịch đảo	Cùng công cụ ở D/21, nhưng đảo ngược mục tiêu: ở đây ta muốn tính nhanh, ở đó người ta dựa vào việc <i>bài toán ngược</i> không tính nhanh được.
Tài chính định lượng	Kỳ vọng và định giá theo giá trị kỳ vọng	Cùng khái niệm ra đời năm 1654 cho cùng loại câu hỏi “ván này đáng giá bao nhiêu”. Khác: tài chính phải quan tâm cả phương sai và rủi ro đuôi, còn phân tích thuật toán trung bình thường chỉ cần kỳ vọng — trừ khi có hạn kỳ, lúc đó đuôi mới là thứ quyết định.
Lý thuyết trò chơi kinh tế	Chiến lược tối ưu, cân bằng	Trò chơi tổ hợp ở mục 6 là trường hợp rất hẹp: hai người, thông tin đầy đủ, tổng bằng không. Kinh tế học tổng quát hơn nhiều nhưng mất đi lời giải sạch kiểu Sprague–Grundy.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì chúng có bản chất khác hẳn. $1 + 2 + 4 + \dots + n$ là cấp số nhân tăng, và mỗi số hạng lớn hơn tổng của tất cả số hạng trước cộng lại, nên tổng bị chặn bởi hai lần số hạng cuối — vẫn là $O(n)$. Còn $1 + 1/2 + \dots + 1/n$ giảm quá chậm: các số hạng không giảm theo cấp số nhân mà theo tỉ lệ nghịch, và tổng của chúng phân kỳ, chỉ có điều phân kỳ chậm như $\ln n$. Hình 1.1 cho thấy điều đó bằng diện tích.
- A2. Đúng vì kỳ vọng là một tổng có trọng số theo xác suất, và phép lấy tổng thì hoán đổi được với phép lấy tổng — không cần bất kỳ giả thiết độc lập nào. Mạnh vì nó tách một bài toán rối thành nhiều bài rất dễ: bạn không cần biết phân phối của tổng, chỉ cần biết xác suất của từng sự kiện nhỏ. Ví dụ số điểm bất động của hoán vị ngẫu nhiên: các biến chỉ báo phụ thuộc chẳng chặt, nhưng mỗi cái có kỳ vọng $1/n$, nên tổng bằng 1.
- A3. Vì kết quả thật thường lớn hơn mọi kiểu số nguyên có sẵn, mà bài toán lại không muốn ép bạn cài số lớn. Lấy dư giữ được cấu trúc cần thiết: cộng, trừ, nhân đều đi xuyên qua phép lấy dư. Chọn số nguyên tố vì khi đó mọi số khác 0 đều có nghịch đảo, nên phép chia — cần cho công thức có giai thừa — vẫn dùng được. Riêng 998 244 353 còn được chọn vì nó có căn nguyên thủy với bậc là lũy thừa lớn của 2, điều kiện để chạy biến đổi Fourier trên số nguyên (D/21).
- A4. Vì nó là một trong số ít công cụ chứng minh sự tồn tại mà không cần chỉ ra vật thể. Sức mạnh nằm ở việc *chọn chuồng* — bước sáng tạo thật sự. Với bài “trong $n + 1$ số từ 1 tới $2n$ luôn có hai số chia hết nhau”, chuồng là “phần lẻ lớn nhất của mỗi số”, và chỉ có n phần lẻ khả dĩ. Chọn được chuồng đúng thì kết luận thành hiển nhiên; toàn bộ độ khó nằm ở bước chọn đó.
- A5. Khi phép chuyển trạng thái là *tuyến tính*, tức trạng thái mới là tổ hợp tuyến tính của trạng thái cũ. Lúc đó n bước là phép nhân ma trận lũy thừa n , tính được bằng bình phương liên tiếp trong $O(\log n)$ phép nhân ma trận. Đó là lý do dãy Fibonacci sau 10^{18} bước tính được còn phần lớn quy hoạch động thì không — và cũng là gợi ý ngược: gấp n lên tới 10^{18} , hãy tìm cách viết truy hồi ở dạng tuyến tính.

NÓI LẠI THẬT ĐƠN GIẢN

Toán trong chương này không phải để chứng minh định lý, mà để trả lời bốn câu hỏi rất đời thường khi bạn viết chương trình.

Câu thứ nhất: “vòng lặp này chạy bao nhiêu lần?” Nếu mỗi lần bạn làm gấp đôi thì tổng công việc chỉ nhiều gấp đôi lần cuối cùng — giống như bạn gấp đôi số tiền tiết kiệm mỗi tháng, tổng tiền cả năm cũng chỉ hơn tháng cuối một chút. Còn nếu mỗi lần bạn chỉ làm bớt đi một phần rất nhỏ, tổng có thể lớn lên chậm mà không bao giờ dừng — đó là tổng điều hoà.

Câu thứ hai: “có bao nhiêu khả năng?” Bốn cách đếm ở mục 2 trả lời gần hết, và điều quan trọng nhất là biết mình đang đếm có thứ tự hay không, có lặp hay không.

Câu thứ ba: “trung bình thì mất bao lâu?” Mẹo duy nhất bạn thực sự cần là: đừng cố tính trung bình của cả cục. Hãy chẻ nó thành nhiều câu hỏi có/không nhỏ xíu, tính xác suất từng cái, rồi cộng. Cộng được kể cả khi chúng ảnh hưởng lẫn nhau — đó là điều bất ngờ và tiện nhất trong cả chương. Câu thứ tư: “số quá lớn thì làm sao?” Lấy dư. Cộng, trừ, nhân vẫn chạy bình thường sau khi lấy dư; chỉ có phép chia là phải làm kiểu khác.

Một câu: Toán ở đây không phải để chứng minh định lý, mà để trả lời “bao nhiêu lần”, “bao nhiêu cách”, “trung bình bao lâu” và “số lớn quá thì sao”.

Chỗ không rút gọn được: Chọn đúng “chuồng” cho nguyên lý chuồng bồ câu, hay chọn đúng biến chỉ báo, vẫn là bước sáng tạo không có công thức.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Chặn cấp số nhân tăng	Tổng $\leq$ hai lần số hạng lớn nhất. Giải thích khẩu hao của mảng động, chi phí xây cây phân đoạn, kỹ thuật nhân đôi.
••• Tổng điều hoà	$H_n \approx \ln n$: phân kỳ nhưng cực chậm. Xuất hiện ở sàng nguyên tố, quicksort ngẫu nhiên, bài sưu tập đủ bộ.
••• Kỳ vọng tuyến tính	$E[\sum X_i] = \sum E[X_i]$ kể cả khi phụ thuộc. Quy trình: viết thành tổng biến chỉ báo $\rightarrow$ mỗi kỳ vọng là một xác suất $\rightarrow$ cộng.
••• Bốn công thức đếm nền	Có thứ tự không lặp $n!/(n-k)!$; không thứ tự không lặp $\binom{n}{k}$; có lặp có thứ tự n^k ; có lặp không thứ tự $\binom{n+k-1}{k}$.
••• Bốn quy tắc modulo	Cộng/trừ/nhân đi xuyên qua phép lấy dư; chia phải dùng nghịch đảo; lũy thừa bằng bình phương liên tiếp; coi chừng tràn khi nhân hai số cỡ 10^9 .
•• Đẳng thức Pascal	$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ — “phần tử thứ n được chọn hoặc không”; khuôn mẫu của mọi lập luận chia trường hợp trong quy hoạch động.
•• Dãy 1, 2, 5, 14, 42	Số Catalan — dãy ngoặc đúng, cây nhị phân, chia đa giác thành tam giác. Gặp dãy này khi tính tay thì bài có cấu trúc Catalan.
•• Ba bất đẳng thức đuôi	Markov (chỉ cần không âm, yếu nhất); Chebyshev (cần phương sai); Chernoff (cần độc lập, chặn giảm theo hàm mũ).
•• Khi nào lũy thừa ma trận dùng được	Khi truy hồi tuyến tính; cho $O(d^3 \log n)$ với n tới 10^{18} . Cũng dùng để đếm đường đi độ dài k trong đồ thị.
•• Stirling và hệ quả	$\log_2(n!) = \Theta(n \log n)$ — cơ sở trực tiếp của cận dưới $\Omega(n \log n)$ cho sắp xếp so sánh.
•• Nguyên lý chuồng bồ câu	Chứng minh tồn tại mà không cần chỉ ra vật thể; toàn bộ độ khó nằm ở bước chọn chuồng.
• Sprague–Grundy	Trò chơi tách thành phần độc lập: giá trị chung là XOR các giá trị Grundy; Nim là trường hợp riêng.
• Cơ sở tuyến tính trên $\mathbb{F}_2$	Cho phép hỏi “số này có là XOR của một tập con không” và “XOR lớn nhất là bao nhiêu” trong $O(n \log C)$.

••• phải thuộc •• nên nắm • biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Vì sao phân tích quicksort ngẫu nhiên lại ra tổng điều hoà?

| Vì ta đếm bằng biến chỉ báo cho sự kiện “hai phần tử ở vị trí i và j trong thứ tự đã sắp có được so sánh với nhau hay không”. Chúng chỉ được so sánh nếu một trong hai được chọn làm chốt trước mọi phần tử nằm giữa chúng, xác suất là $2/(j-i+1)$. Cộng theo mọi cặp thì mỗi khoảng cách d đóng

góp cỡ $n \cdot 2/(d+1)$, và tổng theo d chính là tổng điều hoà — cho $2n \ln n$. Đây là ví dụ mẫu mực cho quy trình ba bước ở mục 3.

2. Khi nào tôi nên đếm bù thay vì đếm trực tiếp?

| Khi điều kiện “tốt” có dạng “không có cái nào vi phạm” còn điều kiện “xấu” dễ mô tả. Ví dụ đếm hoán vị không có điểm bất động thì khó, nhưng đếm hoán vị có ít nhất một điểm bất động lại làm được bằng bao hàm–loại trừ. Dấu hiệu nhận biết trên đề: có chữ “không”, “ít nhất một”, “khác nhau đôi một”.

3. Tôi luôn nhầm giữa $\binom{n+k-1}{k}$ và $\binom{n}{k}$. Có cách nhớ nào không?

| Hãy nhớ mô hình vật lý thay vì công thức. Chia k viên kẹo giống nhau cho n đứa trẻ: xếp k viên kẹo thành hàng và cắm $n-1$ vách ngăn vào; mỗi cách sắp xếp k viên và $n-1$ vách là một cách chia, tổng cộng $\binom{n+k-1}{k}$ cách. Còn $\binom{n}{k}$ là chọn k đứa trẻ trong n đứa. Hai bài khác nhau ở chỗ vật được chia có phân biệt hay không.

4. Chernoff và Chebyshev khác nhau trên thực tế bao nhiêu?

| Rất nhiều, và khoảng cách lớn dần theo số phép thử. Chebyshev cho xác suất lệch giảm theo $1/t^2$; Chernoff cho giảm theo e^{-t} . Với 100 lần tung đồng xu, hỏi xác suất được từ 70 mặt ngửa trở lên: Chebyshev cho một chặn lỏng cỡ vài phần trăm, Chernoff cho chặn nhỏ hơn nhiều bậc. Cái giá là Chernoff đòi các biến độc lập, còn Chebyshev chỉ cần phương sai — nên khi có phụ thuộc thì Chebyshev vẫn dùng được còn Chernoff thì không.

5. Tại sao $10^9 + 7$ lại phổ biến đến thế?

| Ba lý do thực dụng. Nó là số nguyên tố, nên mọi số khác 0 có nghịch đảo. Nó nhỏ hơn 2^{30} , nên tích hai phần dư nhỏ hơn 2^{60} , vừa trong số nguyên 64 bit có dấu. Và nó đủ lớn để hai kết quả khác nhau hiếm khi trùng phần dư một cách tình cờ. Còn 998 244 353 được chọn khi bài cần biến đổi Fourier trên số nguyên, vì $998\,244\,353 = 119 \cdot 2^{23} + 1$ có căn nguyên thủy bậc 2^{23} .

6. Nếu tôi kém toán, tôi có bị chặn trần trong việc luyện thuật toán không?

| Không bị chặn ở phần lớn nội dung. Cấu trúc dữ liệu, đồ thị, quy hoạch động, tham lam — chiếm phần lớn bài tập và phỏng vấn — chỉ cần đúng những gì trong chương này, mà phần lõi chỉ có ba mẫu ở khối *Cần nhớ*. Toán trở thành rào cản thật sự ở ba nơi hẹp: số học nâng cao (D/21), một số bài đếm tổ hợp khó, và phân tích thuật toán ngẫu nhiên tinh vi. Chiến lược hợp lý là học ba mẫu lõi thật chắc rồi bổ sung theo nhu cầu, chứ không học toán trước rồi mới bắt đầu.

7. Cách nhanh nhất để nhận ra một dãy số quen thuộc khi tính tay ví dụ nhỏ?

| Tính năm tới sáu số hạng đầu rồi so với vài dãy thường gặp: 2^n ; $n!$; Fibonacci 1,1,2,3,5,8; Catalan 1,1,2,5,14,42; số Stirling; số Bell 1,1,2,5,15,52 (chú ý dễ nhầm với Catalan ở ba số đầu). Nếu không khớp, hãy thử tra trong từ điển dãy số nguyên trực tuyến. Nhưng nhớ: nhận ra dãy mới là gợi ý, vẫn phải chứng minh vì sao bài của bạn cho ra dãy đó.

8. Có nên học hàm sinh không?

| Chỉ khi bạn đi sâu vào các bài đếm. Hàm sinh là công cụ đẹp và mạnh cho việc giải truy hồi và đếm cấu trúc, nhưng trong phần lớn bài tập thực tế thì cây đệ quy và quy hoạch động đã đủ. Xếp nó vào loại “biết là có, học khi gặp bài thật cần” — và khi đó *Concrete Mathematics*[1] là nguồn đúng.

■ Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
• Tính tổng $\sum_{i=1}^n i$, $\sum 2^i$, $\sum 1/i$ và xác định bậc của từng cái	Tự luyện	Mục tiêu là phản xạ: nhìn vòng lặp ra ngay bậc, không phải nhớ công thức.
• Tính $\binom{n}{k} \bmod p$ cho n tới 10^6 bằng bảng giai thừa và nghịch đảo	Chương D/21	Tiền xử lý giai thừa và nghịch đảo giai thừa một lần; mỗi truy vấn $O(1)$.

Bài	Nguồn	Điều cần nhận ra (tiếp)
• Tính số điểm bất động trung bình của hoán vị ngẫu nhiên bằng mô phỏng, so với lý thuyết	Thực nghiệm	Kết quả bằng 1 với mọi n — hãy tự kiểm chứng rồi giải thích bằng kỳ vọng tuyến tính.
•• Đếm số dãy ngoặc đúng độ dài $2n$ bằng hai cách: quy hoạch động và công thức Catalan	AtCoder DP, CSES[2]	Hai cách phải khớp; nếu lệch, gần như luôn do quên trường hợp biên $n = 0$.
•• Tính Fibonacci thứ 10^{18} theo modulo bằng lũy thừa ma trận	Bài tập kinh điển	Chú ý thứ tự nhân ma trận và tràn số khi nhân hai phần tử.
•• Chứng minh trong $n + 1$ số bất kỳ từ 1 tới $2n$ luôn có hai số mà số này chia hết số kia	Bài tập kinh điển	Chuồng là “phần lẻ lớn nhất”; toàn bộ bài nằm ở bước chọn chuồng.
•• Bài sưu tập đủ bộ: cần trung bình bao nhiêu lần mở gói để có đủ n loại thẻ	Bài tập kinh điển	Chia thành n giai đoạn, mỗi giai đoạn là biến hình học; tổng cho nH_n .
••• Chứng minh quicksort ngẫu nhiên có kỳ vọng $2n \ln n$ so sánh	[3]	Biến chỉ báo cho từng cặp, xác suất $2/(j - i + 1)$; đây là bài tập kinh điển nhất về kỳ vọng tuyến tính.
••• Dựng cơ sở tuyến tính trên $\mathbb{F}_2$ và tìm XOR lớn nhất của một tập con	Bài tập thi đấu kinh điển	Khử Gauss trên bit; sau khi có cơ sở, thuật toán tham lam từ bit cao xuống bit thấp.
••• Tính giá trị Grundy cho một trò chơi lấy sỏi có luật lạ, tìm quy luật	Bài tập kinh điển	Tính tay các thế nhỏ, lập bảng, tìm chu kỳ — đúng quy trình heuristic số một ở A/01.
••• Dùng bao hàm–loại trừ đếm số hoán vị không có điểm bất động (số mất trật tự)	Bài tập kinh điển	Kết quả xấp xỉ $n!/e$; một kết quả đẹp và bất ngờ đáng tự dẫn ra.

Kết chương và đọc tiếp

Chương này khép lại phần A. Nếu ba chương trước cho bạn quy trình, thước chi phí và thước tính đúng, thì chương này là cái hộp dụng cụ đặt cạnh bàn — không cần thuộc hết, chỉ cần biết trong đó có gì và mở đúng ngăn khi cần. Ba mẫu quay lại nhiều nhất, nhắc lại lần cuối: cấp số nhân bị chặn bởi hai lần số hạng lớn nhất, tổng điều hoà xấp xỉ $\ln n$, và kỳ vọng tuyến tính không cần độc lập.

Từ đây, phần B đổi câu hỏi. Bốn chương vừa rồi hỏi “tôi nghĩ thế nào, và làm sao biết mình đúng”. Bảy chương tiếp theo hỏi một câu rất khác: *dữ liệu nên nằm ở dạng nào để thao tác mà tôi lặp lại nhiều nhất trở nên rẻ*. Bắt đầu bằng thứ đơn giản nhất và cũng bị đánh giá thấp nhất — mảng, cùng những kỹ thuật hai con trở biến $O(n^2)$ thành $O(n)$ (B/06).

Nguồn

- [1] R. L. Graham, D. E. Knuth, and O. Patashnik, *Concrete Mathematics: A Foundation for Computer Science*, 2nd ed. Addison-Wesley, 1994.
- [2] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [3] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge University Press, 1995.